DECISION PACKAGE

Design session — the combined decision set

desk-standard · 2026-07-20 · Rebooted to fold in the desk-platform stream.
This deck runs the combined session: rule each decision, or defer it explicitly.

Where we stand, and how to read this

The briefs are written, reviewed, and signed off — and the parallel desk-platform stream just merged in. What is left is the rulings.

- Eight reviewed decision briefs, grounded in five line-cited evidence dossiers
- The desk-platform stream migrated here — it ruled "grow deskkit", so its decisions bind this repo
- A new leading decision D0 fixes the frame; the platform's four open questions joined the scope
- Each slide is one decision: the question, the options, the evidence headline
- The full briefs are the deep material — this deck is the map, not the territory
- Every ruling becomes an ADR in docs/decisions, cited where it binds

The decision map — rule in this order

Frame first, then model, workflow, surfaces — the later decisions consume the earlier rulings.

- D0 Platform frame (new) — truth regime, element-model track, platform rulings
- D1 Pointer grammar — the foundation; D2 builds on it
- D2 Typed cross-references — the reference primitive
- D3 Item-type validation — small, self-contained
- D4 Findings lifecycle + adoption log
- D5 Agent contract and parity — the headline; consumes D0–D4
- D6 Prompt governance — the mechanism behind D5's instruction-source call
- D7 Spec vs reality on the TypeScript boundary — carries your note
- D8 Identity and hygiene — promoted from backlog at your sign-off

FRAME · RULE FIRST

D0 — The platform frame (new at the reboot)

The parallel stream designed the platform top-down and ruled "grow deskkit" — its decisions land in this repo. D0 reconciles the two streams before anything else is ruled.

- (a) Ratify the platform rulings: grow deskkit; the persona bundle as the v1 proof surface; the non-negotiables
- (b) The truth regime — files-are-truth stands now; "the database becomes truth" is a direction needing a future, named trust gate
- (c) The element model (three planes, beefed spine) proceeds as the schema v2 track — mechanisms ruled here, vocabulary there
- (d) The platform's four open questions join this session (next-to-last slide)
- Why it leads: four decisions (D2, D4, D6, D8) derive their criteria from whichever regime (b) picks

MODEL

D1 — What is a pointer allowed to be?

Work items point at documents. The accepted forms are implemented but specified nowhere — and the spec promises URL pointers that the shipped default gates reject today.

- Option A — ratify today's behavior: the section marker stays advisory; write the spec, build nothing
- Option B — real inside-the-file addressing via explicit anchor ids that survive renames
- Option C — tolerant section resolution that falls back to whole-file when a heading moved
- Option D — fold the grammar into D2's typed reference (rule D2 first)
- Evidence: gates deliberately ignore the section suffix so renames cannot break transitions
- Review proof: the URL contradiction bites the shipped decision and task default gates, not an edge
- A ruling touches: the gate validator, the PM spec's pointer prose, and the default gate seeds

MODEL

D2 — Does "graduated to: 42" become a real reference?

A graduation marker is a plain string with no record of which repo it means. The qualifier can only come from the per-desk profile — so resolution is desk-relative by construction, and shipped code may bake in no default.

- Option 1 — field-local hygiene only; no shared primitive
- Option 2 — one shared reference type; the repo qualifier resolved at read time, never stored
- Option 3 — one shared reference type; the qualifier resolved and frozen at write time
- Option 4 — specify the contract now, migrate the fields later
- The hard test: the store must rebuild from disk — derived fields migrate cheap, caller-supplied fields need real migrations
- A ruling touches: the shared schema contract, both lanes' validators, and the graduation scanner

MODEL

D3 — May a typo'd item type skip every gate?

Creating a work item never checks its type. An unknown type matches no gate rules, so the item advances through every phase ungated — re-proven from source; the checker exists but guards only gate configuration.

- Option A — validate at creation, hard reject an unknown type
- Option B — validate at creation, warn but allow
- Option C — rule the ungated advance deliberate, and document it in the spec
- A ruling touches: one code point (item creation) for A and B; documentation only for C
- Platform note: rule the checking mechanism here — the type list itself is the v2 element model's to define

MODEL + WORKFLOW

D4 — Finish the findings lifecycle; decide what the adoption log is for

Dispositions shipped in the bug floor. The residue: a "dismissed" state no code can set, counts that disagree between commands, no record of who or why — and an adoption log with five of six event kinds never written.

- Option 1 — close the lies, build nothing new
- Option 2 — provenance fields on the finding; shrink the adoption log
- Option 3 — wire the adoption log as the real activity ledger
- Option 4 — kill the adoption log; full provenance columns instead
- Review proof: the log has live readers (CLI, MCP, TUI) plus a desk-guard role — killing it is not free
- Only you can state: must who-disposed-and-why survive a store rebuild?

SURFACE · THE HEADLINE

D5 — One integration contract, two agents

Your symmetry concern, framed: one contract — persona instructions, tool mount, wake layer, write-gate policy — that the librarian and PM agents each instantiate. Deliberate differences survive as documented parameters; accidents become debt.

- Phase 0 widened it: unclaimed tools exist on every surface, not just the PM mount
- The librarian's own prompt is stale independent of PM; three of four TypeScript tools have no skill
- PM import and the admin console are claimed by no persona at all
- The fail-loud precedent from the mcp-serve fix applies to every future mount
- Platform pre-answer: the persona bundle is the v1 proof surface — one desk persona, or two composed?
- Consumes D0 through D4 — the frame and model rulings say what surfaces must expose

SURFACE · THE HEADLINE

D5 — The four calls inside it

One contract framing on top; four sub-decisions underneath, each with its own options in the brief.

- (a) Claude Code packaging — ship a librarian bundle, rule CLI-and-TUI-first deliberate, or a minimal mount-plus-hook
- (b) Mounts — per-module servers, one shared mount with tool-level gating, or document the ride-along as policy; give import and the admin console an owner either way
- (c) The in-binary loop — exclude the PM tools it has no instructions for, or compose it a PM prompt; fix the stale librarian prompt regardless
- (d) Instruction sources — the contract names one source per surface here, or hands the whole question to D6

SURFACE

D6 — Which copy of the agent's instructions wins?

Instructions live in three places — compiled into the binary, editable database rows, plugin markdown — with no rule about which wins. Review proved a database edit vanishes if the store is rebuilt.

- Option A — git is truth: the database row is a cache; edits ephemeral by rule
- Option B — the database is truth: shipped text is a seed; edits are durable personalization
- Option C — documented split: shipped persona git-canonical; customization an explicit overlay
- Option D — status quo, documented as a deliberate per-lane split
- Binding: runtime edits are personalization — never flowing back into shipped artifacts
- Rule for the files-are-truth regime (D0) — say what "reset to shipped" means, and what flips at the gate

SURFACE · CARRIES YOUR NOTE

D7 — What does the TypeScript boundary promise?

The spec reads as if the plugin's TypeScript server can call librarian tools; it ships exactly four profile tools. Review judged the spec sentence ambiguous — leaning "client caller", not a flatly broken promise.

- Your note leans Option B; the platform routes agents through the Go server — B composes as a TypeScript proxy of deskkit
- Option A — amend the spec to match shipped reality
- Option B — extend the boundary to deliver librarian capabilities (proxy design becomes a work item)
- Option C — rule only where tool-surface truth lives; hand amend-versus-extend to D5
- Option D — defer entirely
- Also rule where tool-list truth lives: spec prose, the tool-surface doc, or generated with a drift guard

MODEL · PROMOTED AT SIGN-OFF

D8 — Identity and hygiene: three ruled-in gaps

You promoted these from pull-only backlog to full decisions this morning. Each is independent; each has a leave-it option.

- (a) Rename identity — today a rename discards history. Leave it; a frontmatter id; checksum-based inference; or delegate to git. Whatever wins must survive a store rebuild.
- (b) The entity_type collision — a column and an unrelated schema enum share a name. Leave it; rename the column (migration-cheap); or rename the enum (touches live desks' frontmatter).
- (c) Silent 5000-character text caps — leave them; a one-shot sweep setting explicit sizes; or the sweep plus a guard that stops new unbounded fields.

MERGED SCOPE

The platform's four questions — now yours to answer here

These were waiting on you on the other desk; the reboot moves them into this session. Light evidence, owner-preference calls.

- Q1 — is "goal" the right unit, or OKR-style objective plus key-results?
- Q2 — keep the workstream tag (product / engineering / pm) alongside the three planes, or drop it?
- Q3 — research shape: the review argues a loop (brief, plan, gather, synthesize, re-enter via open questions), not a waterfall
- Q4 — which deferred outputs matter first: exec summary, stakeholder update, demo prep?
- Context: the element model failed parts of its completeness review — these answers steer its revision, they don't finalize it

What the review proved — five facts that move option weights

After the briefs were written, every claim that went beyond the dossiers was re-derived from source by an independent reviewer. All five survived; two got sharper.

- The adoption log is read on three surfaces and guards desk collisions — weighs against D4's kill option
- The URL-pointer contradiction bites the shipped default gates today — raises D1's urgency
- The TypeScript boundary ships four tools; the spec sentence is ambiguous, not broken — softens D7's amend case
- Runtime prompt edits are ephemeral across a store rebuild — D6's premise is safe to rule on
- Unknown item types advance ungated, exactly as briefed — D3's mechanism is exact

After the rulings

Rulings become records, records become the build plan — nothing ships from this deck directly.

- Each ruling lands as an ADR in docs/decisions, append-only, cited where it binds
- Spec deltas drafted: the librarian spec, the PM spec, and the shared schema contract
- Then the build plan: epics and issues with acceptance criteria and gate labels — deliverables and parallelism, never timelines
- Every model change is a forward migration; your live desks migrate, nothing is edited in place
- The element-model work (R3 to R4) re-homes here as the schema v2 track
- The PM default-on lane (#83) slots wherever the rulings put it

THE ASK

Rule the set: D0 fixes the frame, D1 through D8 follow, the platform's four questions close it. An explicit deferral is a valid ruling; an open question is not. Every decision leaves the session with an owner and a record.